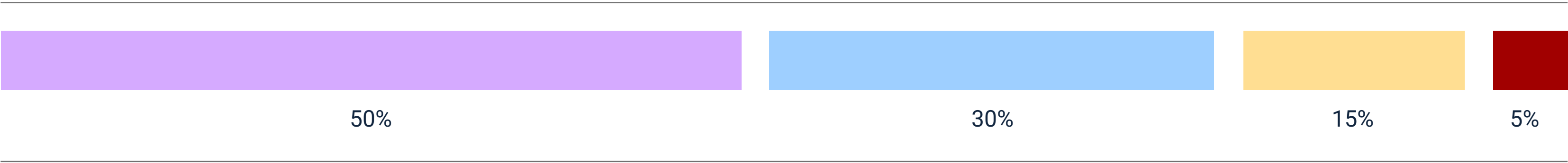
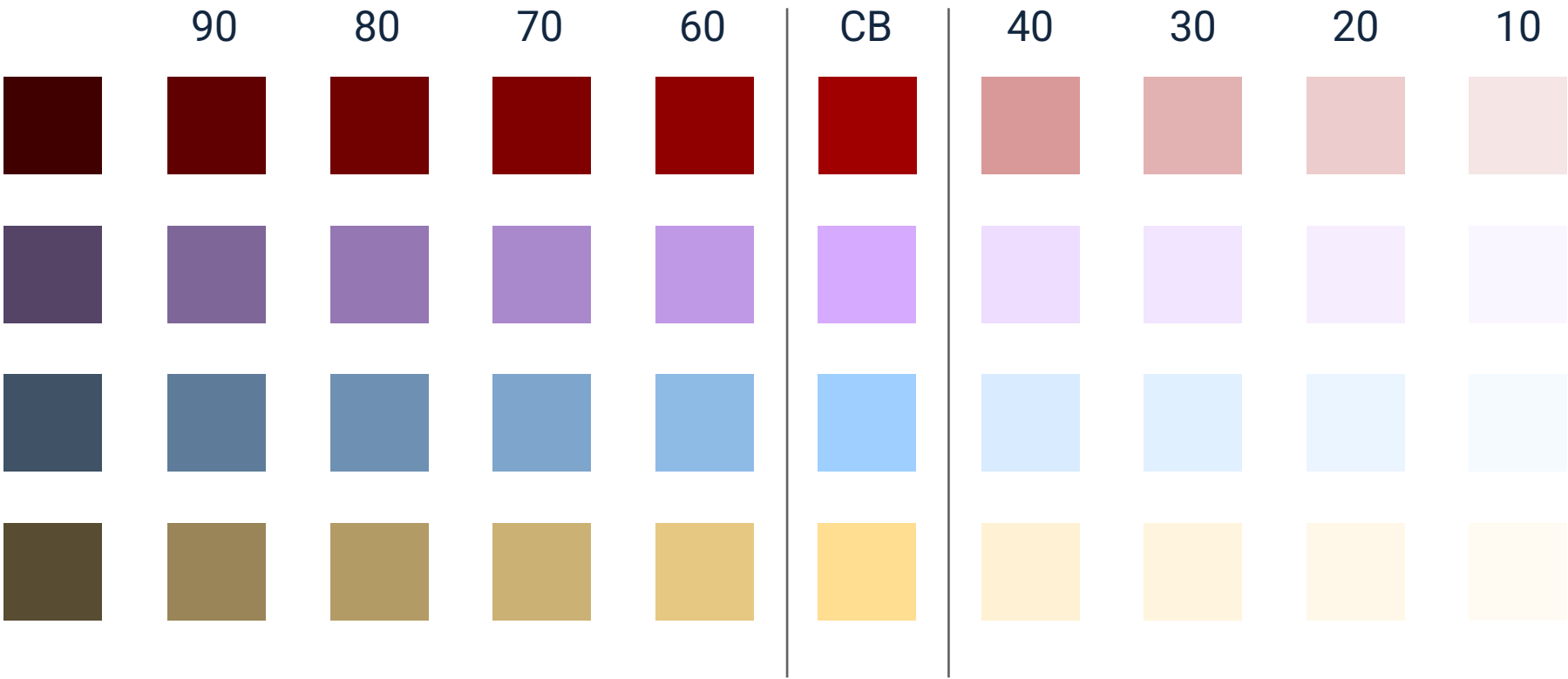
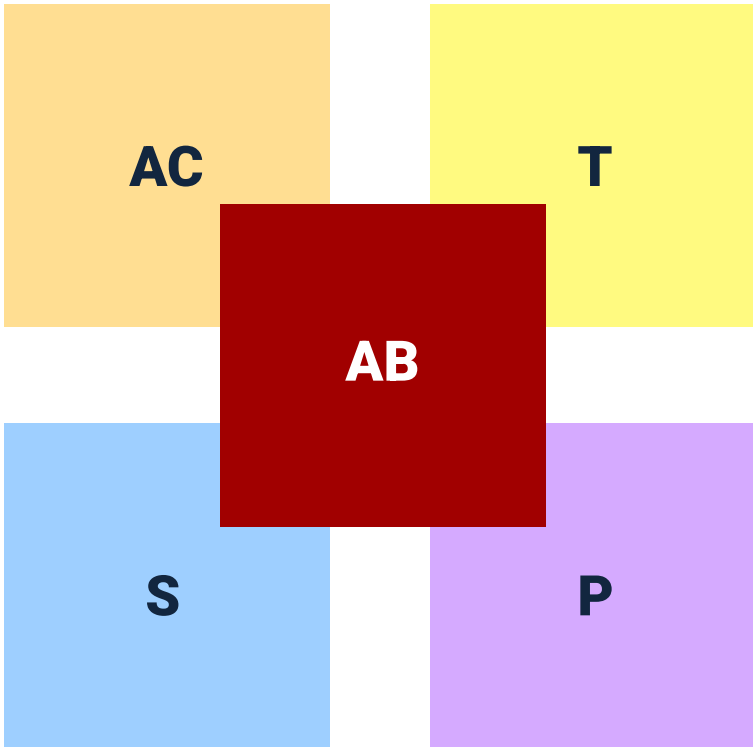


Fundamentos para el desarrollo de aplicaciones web

Este componente es una guía técnica y pedagógica para desarrollar competencias en maquetación, estructuración y despliegue de soluciones web. En diferentes unidades temáticas aborda fundamentos de arquitectura web, comunicación cliente-servidor (*client-server*), desarrollo *frontend* y *backend*, especificación de requerimientos técnicos y analítica de información, aplicando estándares de interoperabilidad y protocolos de red para la mejora continua del *software*.

Ver más >



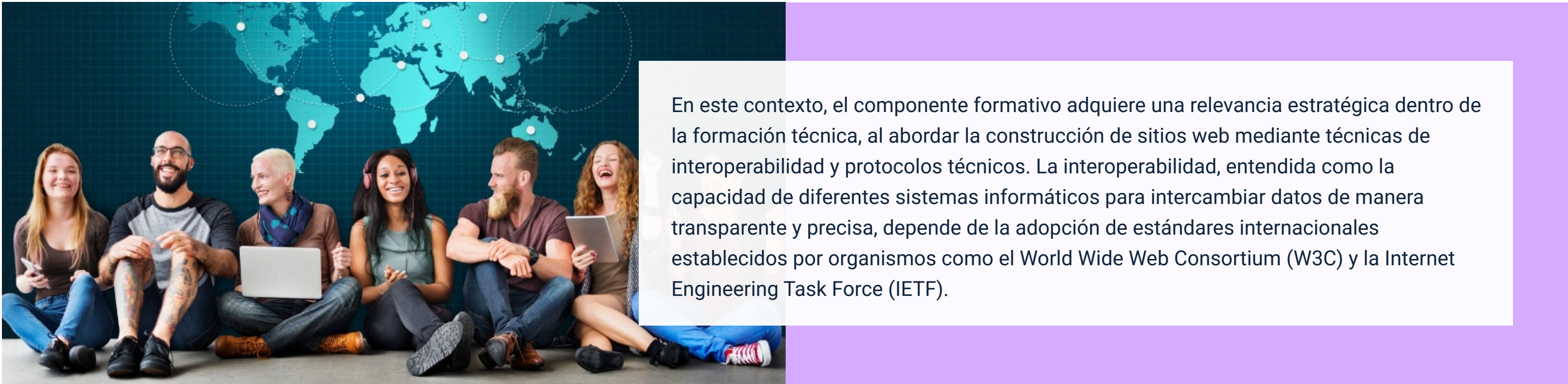
1- Primario	#D5AAFF	2- Secundario	#9ECFFF	3- Acento contenido	#FFDE92	4- Acento BOTÓN	#A10000
1- VC	#F1E2FF	2- VC	#DFEFFF	3- VC	#FFEBBE	4 - VC	#F9E7E7
1- V	#DDA2FF	2- V	#7DBFFF	3- V	#FBB03F	5- Terciario	#FFFA80
1- VO	#62249E	2- VO	#19538D	3- VO	#7A4F1E	5- VC	#FFFDCE

VC = Variable clara
V= Variable
VO= Variable Oscura

i

Introducción

En el panorama tecnológico contemporáneo, el desarrollo de aplicaciones y sitios web ha dejado de ser un proceso meramente estético para convertirse en una disciplina de ingeniería que exige un alto rigor técnico. La masificación de dispositivos con diversas capacidades de procesamiento y resoluciones de pantalla, sumada a la heterogeneidad de los sistemas operativos y navegadores (browsers), plantea el desafío de construir productos digitales que garanticen una experiencia uniforme, accesible y eficiente para cualquier usuario.



El propósito central de este material es dotar a los aprendices de los conceptos, herramientas y metodologías fundamentales para abordar el ciclo de vida del desarrollo web. A lo largo del contenido, se examinan las arquitecturas monolíticas y desacopladas, los protocolos seguros de transferencia de datos, como HTTPS, las mejores prácticas en usabilidad (UX) y accesibilidad (WCAG), así como el uso de herramientas modernas de control de versiones y entornos de desarrollo. Finalmente, se destaca la importancia de evaluar sistemáticamente los resultados mediante analítica de datos, promoviendo una cultura de optimización basada en evidencias e indicadores clave de rendimiento.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:



ESPACIO PARA VIDEO

1 Arquitectura web

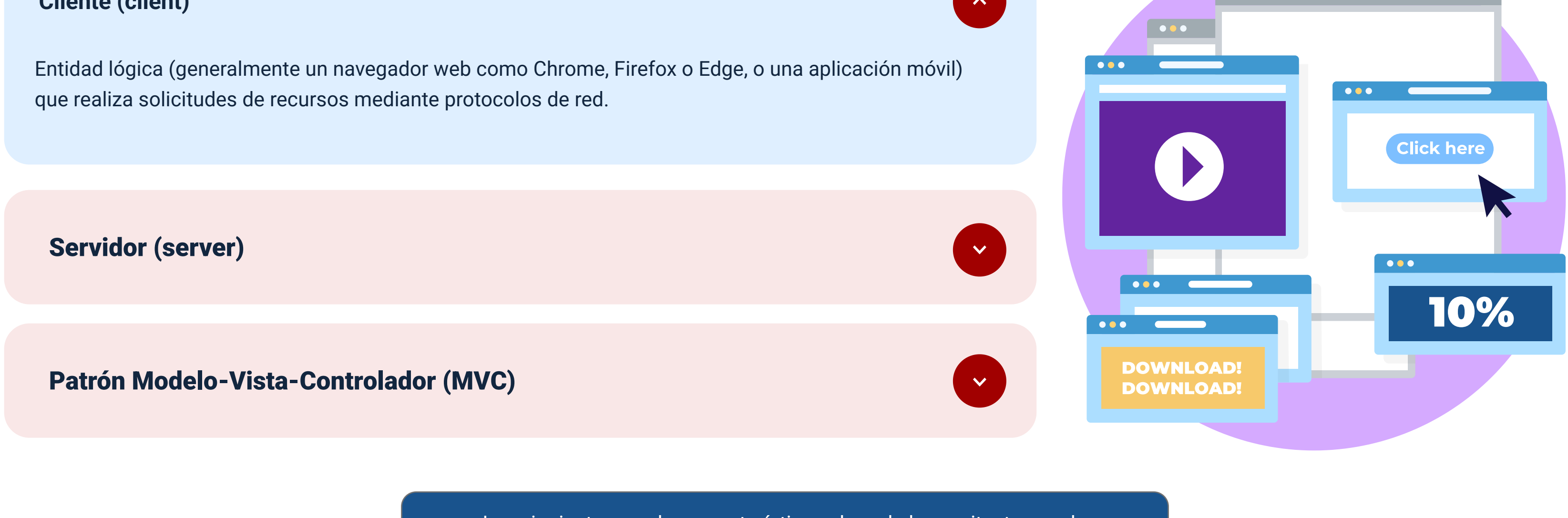


La arquitectura web constituye el conjunto de patrones estructurales, principios de diseño y modelos de abstracción que rigen la interacción entre los componentes de software en una red distribuida. No se limita únicamente a la maquetación o disposición estética del contenido; comprende el diseño conceptual de la infraestructura lógica, la gestión eficiente del flujo de datos, la distribución de servicios y la optimización de recursos informáticos en arquitecturas cliente-servidor (client-server).

1.1 Conceptos y características

El diseño de una arquitectura web robusta exige equilibrar requerimientos técnicos y funcionales para asegurar la continuidad operativa y el rendimiento (performance) del sistema bajo diversas condiciones de demanda.

Para comprender la estructura y el funcionamiento de una arquitectura web, es necesario reconocer los principales elementos que intervienen en la comunicación, el procesamiento de datos y la organización del sistema. A continuación, se presentan los conceptos fundamentales:



Las siguientes son las características clave de la arquitectura web:

- Escalabilidad.** Capacidad de una infraestructura para incrementar sus prestaciones ante una mayor demanda sin degradar la calidad del servicio. Se clasifica en:
 - Escalabilidad vertical (scale up)**
Incremento de recursos físicos (procesador, memoria RAM, almacenamiento) en un único servidor.
 - Escalabilidad horizontal (scale out)**
Adición de múltiples nodos o servidores interconectados bajo un balanceador de carga (load balancer).
- Mantenibilidad y modularidad.** Capacidad de actualizar, depurar o refactorizar módulos de código de forma independiente, reduciendo el riesgo de efectos colaterales en la aplicación.
- Disponibilidad y tolerancia a fallos.** Garantía de funcionamiento continuo del sistema mediante redundancia de componentes, copias de respaldo (backups) y estrategias de failover.
- Desacoplamiento.** Grado de independencia entre los componentes del sistema, lo cual facilita la adopción de tecnologías diversas en cada capa.



El diseño de software moderno prioriza la alta cohesión dentro de cada módulo y el bajo acoplamiento entre módulos, lo que favorece la reutilización del código y la mantenibilidad.

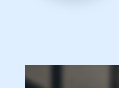
1.2 Tipos de arquitectura web

La selección del tipo de arquitectura condiona la estrategia de renderizado, la latencia de red y la complejidad en el despliegue del software. Para comprender estas diferencias, la siguiente figura presenta de manera comparativa las principales arquitecturas web, destacando su estructura y la forma en que se organizan sus componentes:

Figura 1. Esquema de arquitecturas web principales (monolítica vs. desacoplada con SPA)



A partir de la comparación presentada en la anterior figura, es posible profundizar en las características, el funcionamiento, las ventajas y las limitaciones de los principales tipos de arquitectura web. A continuación, se describen cada uno de estos enfoques:



Arquitectura monolítica (Multi-Page Application - MPA)



En esta arquitectura, el código de la interfaz de usuario, la lógica de negocio y el acceso a la base de datos se compilan y despliegan dentro de una única unidad ejecutable. Cada interacción del usuario implica el procesamiento de una nueva petición HTTP en el servidor y la entrega de un documento HTML completo, generando una recarga visible de la página en el navegador.

- Ventajas:** sencillez en las etapas iniciales de desarrollo e indexación SEO nativa.
- Desventajas:** alto consumo de ancho de banda por la transferencia repetida de activos y acoplamiento estricto.



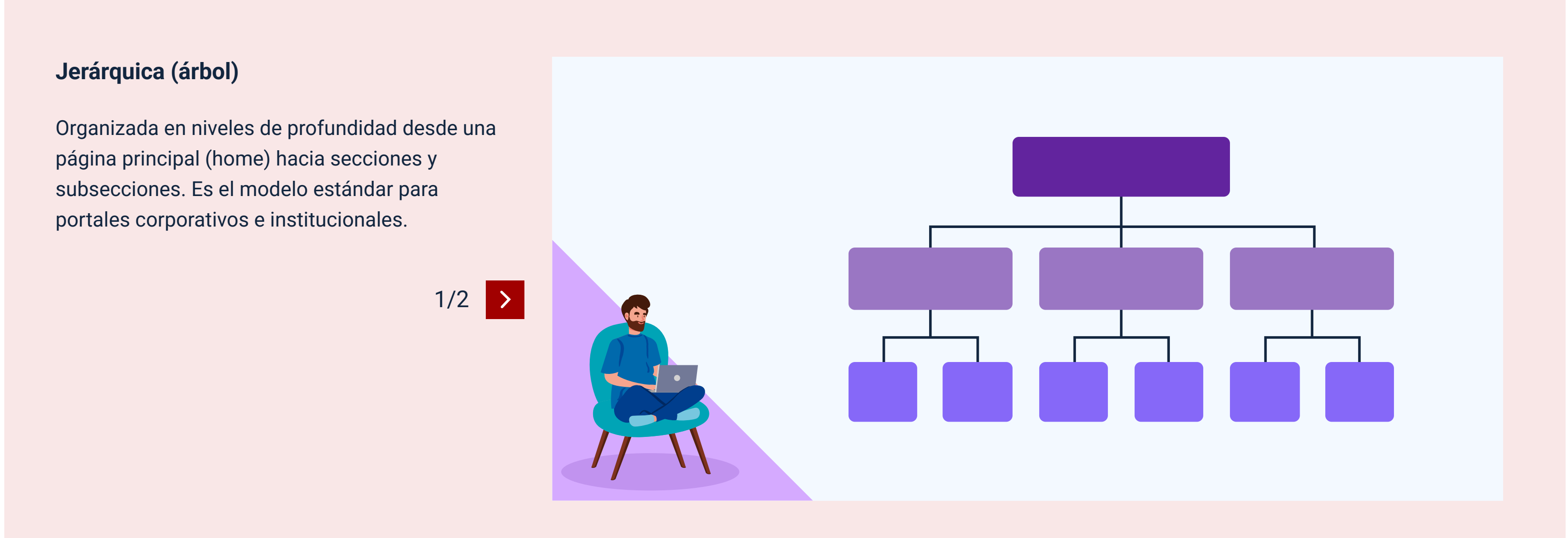
Título ítem 2



Título ítem 3

1.3 Mapas de navegación y procedimiento para diseñar

El mapa de navegación es la representación esquemática de la estructura de la información y los caminos de acceso que el usuario puede seguir dentro del sitio web. Según la forma en que se organizan y relacionan los contenidos, pueden establecerse diferentes estructuras de navegación, cada una adecuada para determinadas necesidades y recorridos del usuario. A continuación, se presentan los principales tipos:



Una vez identificada la estructura de navegación más adecuada, es necesario organizar y representar los contenidos mediante un proceso que permita definir jerarquías, relaciones y recorridos de usuario. Para ello, se propone el siguiente procedimiento:

- Investigación y clasificación (card sorting).** Técnica participativa para agrupar y categorizar los contenidos de acuerdo con la mentalidad del usuario final.
- Definición de la arquitectura de información (AI).** Creación del inventario completo de contenidos y definición de las jerarquías.
- Diagramación del mapa de sitio (sitemap).** Elaboración del flujo visual que especifica la relación padre-hijo de cada URL o pantalla.
- Bocetado de baja fidelidad (wireframing).** Diseño de esquemas de distribución espacial para verificar la ubicación de menús, migas de pan (breadcrumbs) y llamadas a la acción (CTA).
- Validación de usabilidad.** Evaluación de los mapas de navegación para garantizar que el usuario pueda alcanzar cualquier contenido en un máximo de tres clics.

1.4 Técnicas de integración y sintaxis de contenidos web

La construcción del código fuente debe seguir los estándares internacionales del World Wide Web Consortium (W3C), asegurando que la estructura sea interpretable por cualquier agente de usuario. En este proceso, HTML5 permite organizar el contenido mediante etiquetas semánticas que identifican la función de cada bloque dentro del documento, evitando el uso indiscriminado de elementos genéricos como <div> y favoreciendo una estructura más clara y organizada.



Estructuración semántica en HTML5

Para apreciar la aplicación de estas etiquetas, el siguiente código presenta un ejemplo básico de estructuración semántica en HTML5:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Estructura Semántica Profesional</title>
</head>
<body>
  <header>
    <h1>Centro de Formación Técnica</h1>
    <nav>
      <ul>
        <li><a href="#inicio">Inicio</a></li>
        <li><a href="#cursos">Cursos</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="cursos">
      <article>
        <h2>Desarrollo de Aplicaciones Web</h2>
        <p>Contenido explicativo sobre la arquitectura del software...</p>
      </article>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Servicio Nacional de Aprendizaje - SENA</p>
  </footer>
</body>
</html>
```

Una vez definida la estructura semántica del contenido mediante HTML5, es necesario integrar los recursos encargados de la presentación visual y del comportamiento interactivo de la página. Para ello, CSS3 y JavaScript complementan la estructura del documento y permiten mantener una adecuada separación entre contenido, presentación y funcionalidad. Entre las principales técnicas de integración se encuentran:



1.5 Características y licenciamiento de la plataforma



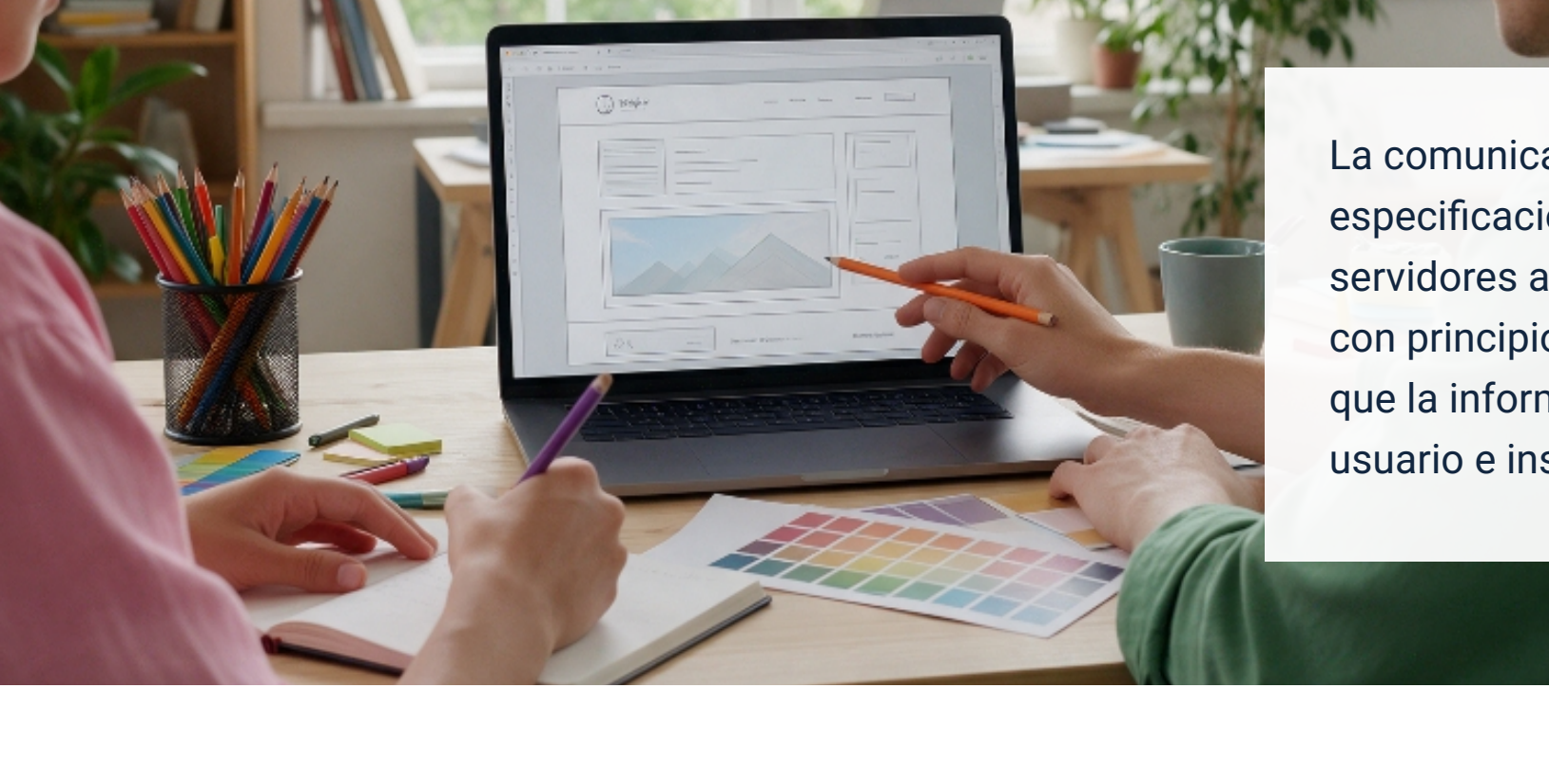
El entorno donde se ejecutan y alojan los contenidos web exige analizar el tipo de licenciamiento técnico y legal para proyectar la viabilidad financiera del proyecto.

El uso de software bajo licencias de código abierto (open source) no excluye la existencia de costos operativos (OPEX), los cuales se desplazan desde el pago de derechos de uso hacia las tareas de infraestructura, auditoría de seguridad y soporte especializado. En este sentido, la selección de una plataforma requiere considerar aspectos como el costo de licencia, el acceso al código, la seguridad, las condiciones de despliegue y la flexibilidad de personalización. La siguiente tabla compara estos criterios en modelos de código abierto y plataformas propietarias o bajo modalidad SaaS.

Tabla 1. Cuadro comparativo de modelos de licenciamiento y plataformas

Criterio de evaluación	Licencia GPL y MIT	Licencia propietaria y SaaS
Costo de licencia.	Gratuita para modificación y distribución.	Pago por suscripción, volumen o transacción.

2 Comunicación y diseño web



La comunicación en el entorno web constituye el conjunto de mecanismos, reglas y especificaciones técnicas que regulan la transferencia de información entre clientes y servidores a través de infraestructuras de red. Este flujo de datos debe complementarse con principios rigurosos de diseño visual, ergonomía digital y usabilidad, garantizando que la información se transmita de manera íntegra, eficiente y accesible para cualquier usuario e instrumento de navegación.

2.1 Técnicas de comunicación

El paradigma fundamental de la arquitectura web se basa en el modelo cliente-servidor (client-server). La interacción entre ambos extremos se realiza mediante peticiones y respuestas estructuradas. De acuerdo con el comportamiento de la interfaz y la sincronización en el intercambio de datos, las técnicas de comunicación se dividen en dos modalidades principales:



2.2 Protocolos de comunicación

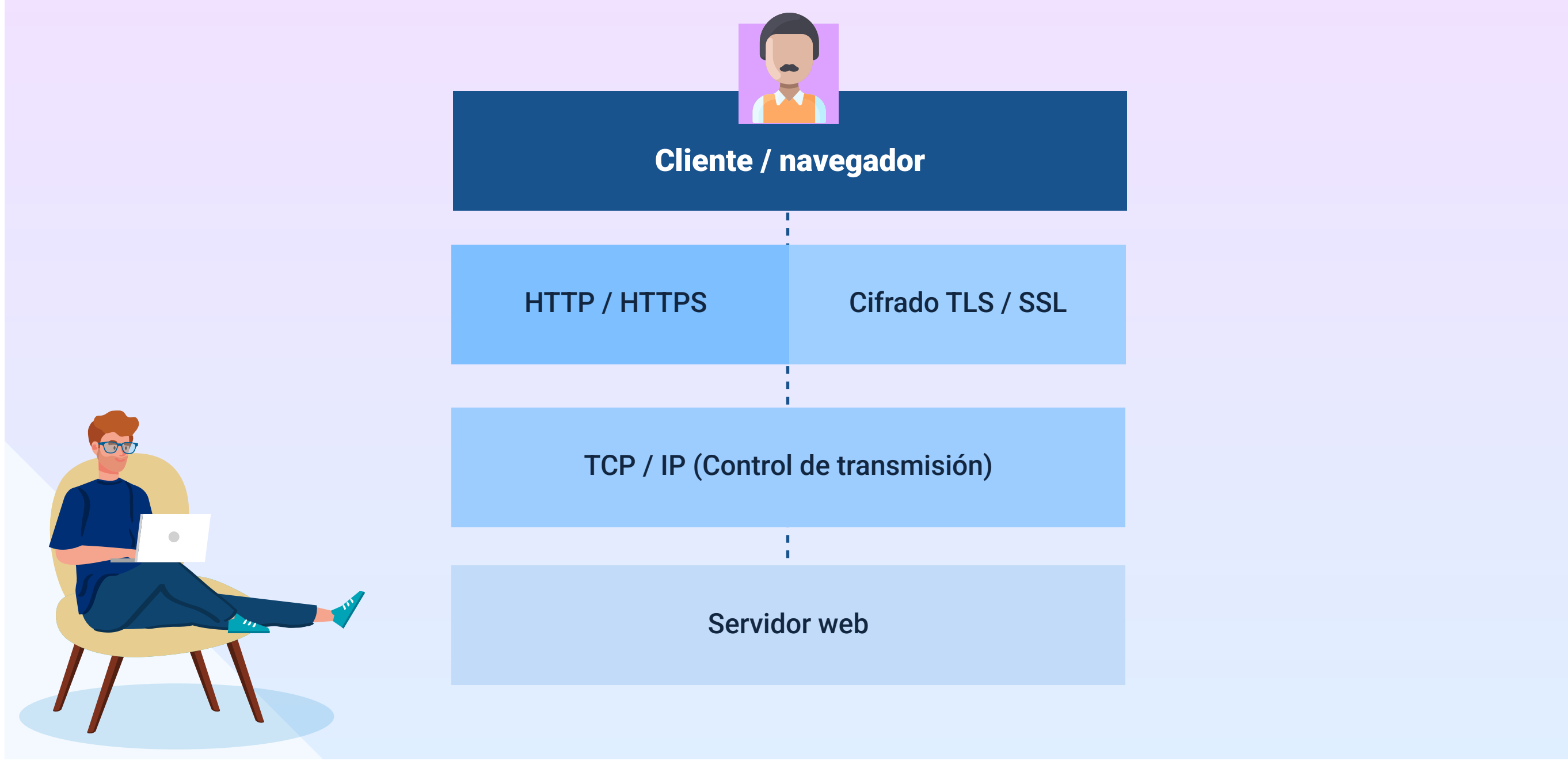
Un protocolo de comunicación es un conjunto formal de reglas, sintaxis y procedimientos que permiten organizar y controlar la transmisión de datos a través de una red. En la comunicación web, estos protocolos establecen la manera en que clientes y servidores intercambian solicitudes y respuestas. Entre los principales protocolos utilizados para este intercambio se encuentran HTTP y HTTPS, explicados a continuación:

A. Protocolo HTTP (Hypertext Transfer Protocol)

HTTP es un protocolo de la capa de aplicación que permite el intercambio de información entre clientes y servidores en la web. Su funcionamiento se basa en un modelo de petición-respuesta: el cliente, generalmente un navegador, envía una solicitud para acceder a un recurso o ejecutar una acción, y el servidor procesa la petición y devuelve una respuesta con la información correspondiente.

Una de sus principales características es que funciona como un protocolo sin estado (stateless), lo que significa que cada petición se procesa de manera independiente y, por defecto, el servidor no conserva información sobre las solicitudes anteriores. Para comprender cómo HTTP se integra con las diferentes capas que intervienen en la transmisión de datos entre el cliente y el servidor. La siguiente figura representa de manera esquemática este proceso de comunicación:

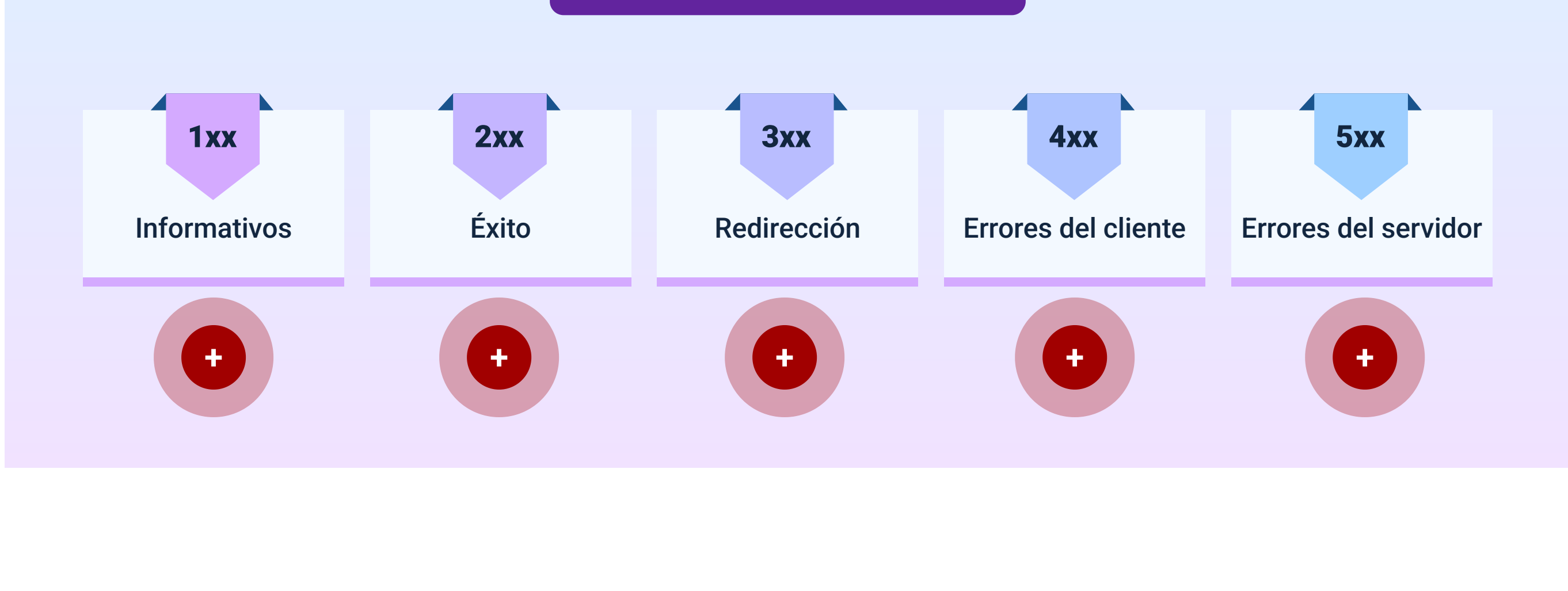
Figura 2. Diagrama de flujo y capas de los protocolos de comunicación



Las peticiones HTTP utilizan diferentes métodos para indicar la acción que el cliente desea realizar sobre un recurso del servidor. Cada método cumple una función específica dentro del intercambio de información y permite distinguir operaciones como consultar, enviar, actualizar o eliminar datos. Entre los métodos más utilizados se encuentran:



Una vez procesada la petición, el servidor devuelve una respuesta que incluye un código de estado HTTP. Este código permite identificar de manera estandarizada el resultado de la solicitud y se organiza en cinco categorías principales según el tipo de respuesta:

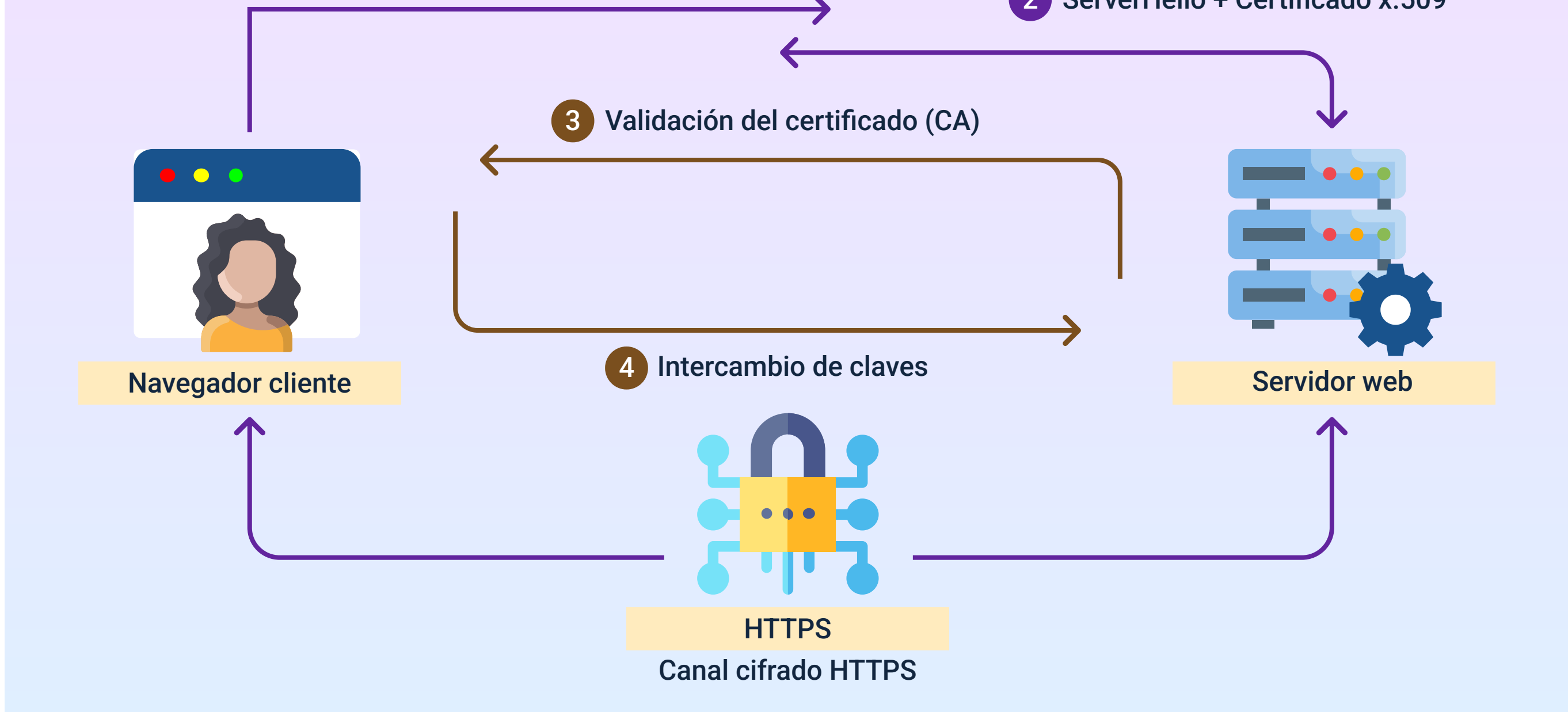


B. Protocolo HTTPS (Hypertext Transfer Protocol Secure)

HTTPS es la implementación segura del protocolo HTTP y utiliza el protocolo Transport Layer Security (TLS), sucesor de Secure Sockets Layer (SSL), para proteger la información intercambiada entre el cliente y el servidor. Este mecanismo permite cifrar los datos transmitidos, verificar la identidad del servidor mediante certificados digitales y preservar la integridad de la información durante la comunicación.

Antes de establecer el canal cifrado, el cliente y el servidor realizan un proceso de negociación en el que intercambian información, validan el certificado digital y establecen los parámetros necesarios para proteger la conexión. La siguiente figura representa las principales etapas de este proceso:

Figura 3. Diagrama de secuencia del apretón de manos HTTPS



• Pilares de seguridad en HTTPS

La seguridad proporcionada por HTTPS se sustenta en tres pilares fundamentales: el cifrado, la integridad y la autenticación. El cifrado de datos protege la información intercambiada entre el cliente y el servidor, de manera que credenciales, datos personales u otra información sensible no puedan ser interpretados fácilmente si son interceptados durante la transmisión.

La integridad de los datos permite verificar que la información no haya sido modificada durante su recorrido entre el cliente y el servidor. Los mecanismos criptográficos utilizados por TLS permiten detectar alteraciones en los datos transmitidos y evitar que información manipulada sea aceptada como válida.

Por su parte, la autenticación permite al cliente verificar la identidad del servidor mediante certificados digitales emitidos por autoridades de certificación (CA) de confianza. Este proceso contribuye a confirmar que la comunicación se establece con el servidor esperado y no con un tercero que intenta suplantarlos.



De esta manera, el cifrado, la integridad y la autenticación actúan de forma complementaria para proteger la comunicación HTTPS y proporcionar un intercambio de información más seguro en las aplicaciones web.

2.3 Características del diseño web

El diseño web moderno se fundamenta en la confluencia entre la usabilidad, la experiencia de usuario (UX), la interfaz de usuario (UI) y el cumplimiento normativo de accesibilidad universal. En este contexto, el diseño web debe responder tanto a la diversidad de dispositivos y tamaños de pantalla como a las necesidades de acceso de los usuarios. Para ello, se consideran dos características fundamentales:

A. Diseño web responsivo (responsive web design)

Consiste en la capacidad de una maquetación para reestructurar sus componentes de manera fluida en función del espacio disponible en pantalla, utilizando hojas de estilo CSS3, rejillas flexibles (css grid), cajas flexibles (flexbox) y reglas de consulta de medios (media queries).



• Enfoque mobile-first

Este enfoque es una estrategia de diseño y desarrollo que prioriza inicialmente la maquetación para dispositivos móviles de pantallas reducidas y agrega complejidad visual y estructural de forma progresiva a medida que aumenta el tamaño de la pantalla (progressive enhancement). En CSS, este enfoque puede implementarse mediante estilos base para dispositivos móviles y consultas de medios (media queries) que modifican la disposición de los elementos en pantallas de mayor tamaño, como se presenta en el siguiente ejemplo:

```
/* Estilos Base: Mobile-First */
.contenedor-principal {
  display: flex;
  flex-direction: column;
  width: 100%;
  padding: 1rem;
}

/* Punto de interrupción (Breakpoint) para tabletas */
@media (min-width: 768px) {
  .contenedor-principal {
    flex-direction: row;
    gap: 2rem;
  }
}

/* Punto de interrupción para pantallas de escritorio */
@media (min-width: 1200px) {
  .contenedor-principal {
    max-width: 1200px;
    margin: 0 auto;
  }
}
```

B. Accesibilidad web (Web Content Accessibility Guidelines - WCAG)

Las pautas de accesibilidad para el contenido web, desarrolladas por el W3C, establecen criterios orientados a garantizar que los sitios web sean utilizables por personas con diversas discapacidades, incluidas las visuales, auditivas, motoras y cognitivas. Estos criterios se organizan en cuatro principios fundamentales, conocidos por el acrónimo POUR:

Perceptible	Operable	Comprensible	Robusto
La información y los componentes de la interfaz deben ser presentados de modo que los usuarios puedan percibirlos (por ejemplo, proporcionando alternativas de texto mediante atributos alt en imágenes).	La interfaz no puede requerir interacciones que el usuario no pueda realizar; toda la funcionalidad debe ser accesible mediante el teclado sin atrapamiento del foco (keyboard trap).	La información y el funcionamiento de la interfaz de usuario deben ser comprensibles y predecibles.	El contenido debe ser lo suficientemente robusto para ser interpretado de forma fiable por una amplia variedad de agentes de usuario, incluyendo tecnologías de asistencia (screen readers).

Para cumplir con los criterios de contraste correspondientes al nivel AA de las WCAG, la relación de contraste entre el texto y su fondo debe ser de al menos 4.5:1 para texto normal y de 3:1 para texto de gran tamaño.

En conjunto, los principios de comunicación, seguridad, adaptabilidad y accesibilidad permiten establecer criterios técnicos para evaluar el funcionamiento y la experiencia de uso de un sitio web. Estos criterios pueden verificarse mediante parámetros relacionados con la seguridad de la red, el tiempo de respuesta, el contraste visual, la adaptabilidad de la interfaz y la navegabilidad. La siguiente tabla sintetiza estos aspectos y presenta algunos parámetros de referencia para su evaluación:

Tabla 2. Directrices técnicas de comunicación y diseño web

Dimensión técnica	Parámetro de evaluación	Estándar / recomendación técnica	Frecuencia de verificación
Seguridad de red.	Certificados de cifrado.	TLS 1.3 y renovación automática mediante Let's Encrypt.	Continuo.

3 Desarrollo web



El desarrollo web abarca la fase de implementación práctica del ciclo de vida del software, integrando la lógica de programación del lado del cliente (frontend), la lógica del servidor (backend), el manejo de bases de datos y la orquestación de herramientas especializadas para la edición, el empaquetado y la integración continua.

3.1 Técnicas de desarrollo web

La construcción de software web se fundamenta en la separación de responsabilidades (Separation of Concerns, SoC), principio que permite organizar los diferentes componentes de una aplicación de acuerdo con las funciones que cumplen. Esta separación facilita distinguir los elementos relacionados con la estructura, la presentación y la interacción del usuario de aquellos encargados de la lógica de negocio, la seguridad y la gestión de los datos. Desde esta perspectiva, el desarrollo web se aborda mediante dos áreas principales:

A. Desarrollo frontend

El desarrollo frontend comprende los componentes de una aplicación web que se ejecutan en el navegador y con los que el usuario interactúa directamente. Su construcción integra la estructura del contenido, la presentación visual y el comportamiento interactivo de la interfaz. Para comprender cómo se desarrollan estos componentes, se presentan las principales técnicas y tecnologías empleadas en el frontend:

Estructuración semántica (HTML5)

Definición de la jerarquía conceptual del documento mediante elementos y etiquetas semánticas que organizan el contenido de forma lógica, accesible y estructurada, facilitando su interpretación por navegadores, motores de búsqueda y tecnologías de asistencia.

Estilizado y maquetación avanzada (CSS3)

Definición del comportamiento sintáctico y estético mediante metodologías como BEM (Block Element Modifier) o arquitecturas CSS atómicas.

Lógica de cliente interactiva (JavaScript ES6+)

Programación imperativa o funcional para gestionar el estado de la interfaz, el procesamiento de eventos, la validación de formularios en el cliente y la manipulación del DOM.

B. Desarrollo backend

El desarrollo backend comprende los procesos que se ejecutan en el servidor y que permiten gestionar la lógica de negocio, la seguridad de acceso, la autenticación de usuarios, el procesamiento de solicitudes y la persistencia de datos. Su implementación se apoya en diferentes tecnologías y enfoques que permiten estructurar y gestionar estas funciones. Entre los principales se encuentran:

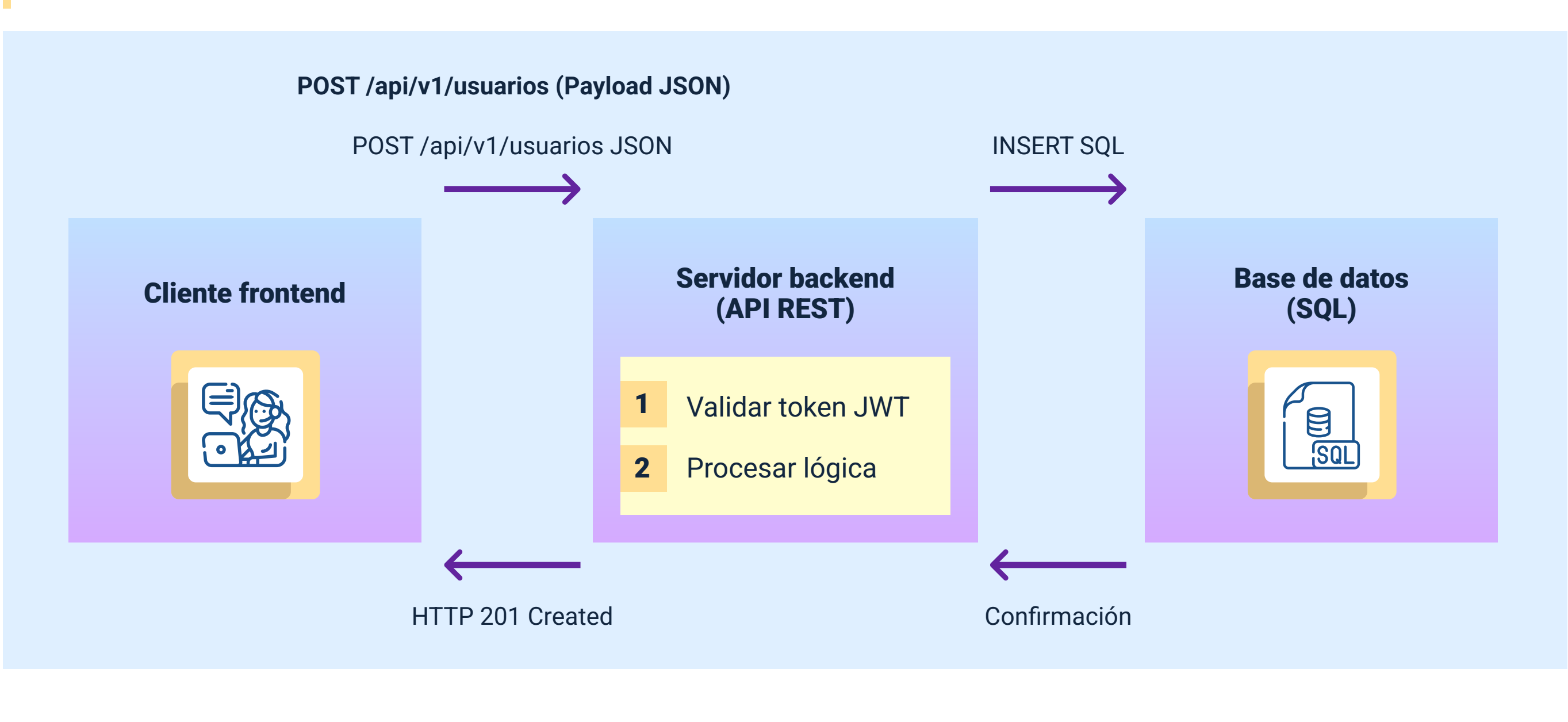
Lenguajes de programación del servidor.

Utilización de lenguajes y entornos como Node.js, Python (Django / FastAPI), PHP, Java (Spring Boot) o C# (.NET Core) para implementar la lógica de negocio, procesar solicitudes, gestionar datos y ejecutar las operaciones requeridas por las aplicaciones web.

Estilo arquitectónico REST (Representational State Transfer).

Diseño de servicios web que exponen recursos mediante URLs univocas y operaciones estándar asociadas a los verbos HTTP, utilizando JSON como formato preferente para el intercambio de información.

La interacción entre estos componentes puede apreciarse en el flujo de una solicitud desde el cliente hacia el servidor, donde la API REST procesa la petición, ejecuta la lógica correspondiente y se comunica con la base de datos para gestionar la información. La siguiente figura representa de manera esquemática este proceso y el intercambio de respuestas entre los diferentes componentes:



3.2 Herramientas de desarrollo web

El flujo de trabajo moderno exige un ecosistema de herramientas especializadas para optimizar la productividad del desarrollador y la calidad del código entregado.

La siguiente es la cadena y flujo de trabajo con herramientas de desarrollo web:

1

Editores de código (VS Code)
Herramientas de edición de código con soporte para resaltado de sintaxis, autocompletado inteligente, refactorización y depuración integrada (p. ej., Visual Studio Code y JetBrains WebStorm).

Control de versiones (Git y GitHub)
Herramientas y plataformas que permiten registrar y gestionar los cambios realizados en el código fuente a lo largo del tiempo, facilitando la colaboración, la trazabilidad y el trabajo mediante ramas (branches).

2

3

Empaquetador (Vite o Webpack)
Herramientas que permiten gestionar y optimizar los recursos de una aplicación durante el proceso de desarrollo y construcción, incluyendo tareas como el procesamiento de dependencias, la transformación del código y la generación de archivos optimizados para su distribución.

Contenedor (Docker)
Tecnología de contenerización que permite empaquetar una aplicación junto con sus dependencias y configuraciones en un entorno aislado y reproducible, facilitando su ejecución consistente en diferentes entornos de desarrollo, prueba y producción.

4

5

Servidor de producción
Entorno en el que la aplicación se despliega y queda disponible para los usuarios finales. Comprende la configuración del servidor, la ejecución de los servicios necesarios y la gestión de aspectos como disponibilidad, seguridad, rendimiento y monitoreo.

Es importante destacar que, dentro de este flujo de trabajo, el control de versiones permite gestionar de manera organizada los cambios realizados durante el desarrollo de una aplicación. Como ejemplo de su aplicación, los siguientes comandos muestran un flujo básico de trabajo con Git, desde la inicialización de un repositorio local y la creación de una rama hasta el registro de los cambios y su envío al repositorio remoto:

```
# Inicializar un nuevo repositorio local
git init

# Crear una nueva rama para una característica específica
git checkout -b feature/autenticacion-jwt

# Registrar cambios en la zona de preparación (Staging Area)
git add .

# Confirmar los cambios en el historial local con un mensaje descriptivo
git commit -m "feat: implementar validación de tokens JWT en el módulo de login"

# Vincular y enviar la rama al repositorio remoto
git push origin feature/autenticacion-jwt
```

3.3 Características de las herramientas

Las herramientas de desarrollo web se diferencian por la función que desempeñan dentro del proceso de construcción de una aplicación y por las características técnicas que aportan al flujo de trabajo. Mientras algunas se orientan al control y trazabilidad de los cambios en el código, otras facilitan la construcción y optimización de los recursos, la estandarización de los entornos, el análisis de la calidad del código o la validación de servicios y APIs.

Estas herramientas también favorecen la automatización y la consistencia de los procesos de desarrollo. Por ejemplo, los sistemas de control de versiones permiten conservar el historial de cambios y apoyar el trabajo colaborativo; las herramientas de construcción procesan y optimizan los recursos de una aplicación; y los contenedores permiten reproducir configuraciones y dependencias en diferentes entornos. A estas se suman herramientas destinadas al análisis estático del código y a las pruebas de APIs, que contribuyen a detectar errores y verificar el comportamiento de los servicios antes de su despliegue.

Por esta razón, la selección de herramientas no depende únicamente de la tecnología utilizada para desarrollar la aplicación, sino también de aspectos como la trazabilidad, la portabilidad, la automatización, la calidad del código y las necesidades de prueba. La siguiente tabla presenta una comparación de algunas herramientas representativas del ecosistema de desarrollo web, organizadas según su categoría, función principal y característica clave:

Herramienta	Categoría	Función principal	Característica clave
Git.	Control de versiones.	Seguimiento de cambios en código.	Arquitectura distribuida e historial inmutable mediante hashes SHA-1.

4 Sitio web

Un sitio web es una colección estructurada de documentos de hipertexto, hojas de estilo, scripts y activos multimedia interconectados de forma lógica, alojados en una infraestructura de servidores y accesibles a través de Internet mediante los protocolos HTTP o HTTPS.



4.1 Definición y características

Para que una solución web alcance condiciones adecuadas de calidad y funcionamiento, debe integrar características relacionadas con su identificación, disponibilidad y capacidad de operar en diferentes entornos tecnológicos. Estos aspectos favorecen el acceso a los recursos, la continuidad del servicio y una experiencia consistente para los usuarios, independientemente del navegador o sistema operativo utilizado. Entre las principales características se encuentran:



Identificación e indexabilidad

Direccionamiento inequívoco mediante nombres de dominio y resolución DNS, junto con una estructura del sitio que facilite su exploración e indexación por los motores de búsqueda.



Disponibilidad operativa

Capacidad del sitio para mantenerse accesible y en funcionamiento durante un periodo determinado, generalmente expresada mediante un porcentaje de disponibilidad (uptime). Los niveles esperados pueden establecerse de acuerdo con los requisitos del servicio y los acuerdos de nivel de servicio (SLA).



Interoperabilidad multiplataforma

Capacidad de un sitio web para mantener un funcionamiento y una experiencia de uso consistentes en diferentes navegadores (cross-browser compatibility), dispositivos y sistemas operativos.

4.2 Tipos y categorías

Los sitios web pueden clasificarse de acuerdo con su propósito, las funcionalidades que ofrecen y el tipo de interacción que establecen con sus usuarios. Esta clasificación permite diferenciar sitios orientados principalmente a la difusión de información de aquellos que incorporan procesos transaccionales, educativos o aplicaciones que proporcionan servicios directamente desde el navegador. Aunque un mismo sitio puede combinar características de varias categorías, es posible identificar algunos tipos principales según su función predominante. Entre ellos se encuentran:



Portales institucionales o corporativos

Orientados a la proyección de la identidad corporativa, la divulgación de información estratégica y la comunicación con sus públicos, facilitando el acceso a servicios, información institucional y canales de atención.

Sistemas de comercio electrónico (E-commerce)

Sistemas de gestión de aprendizaje (LMS)

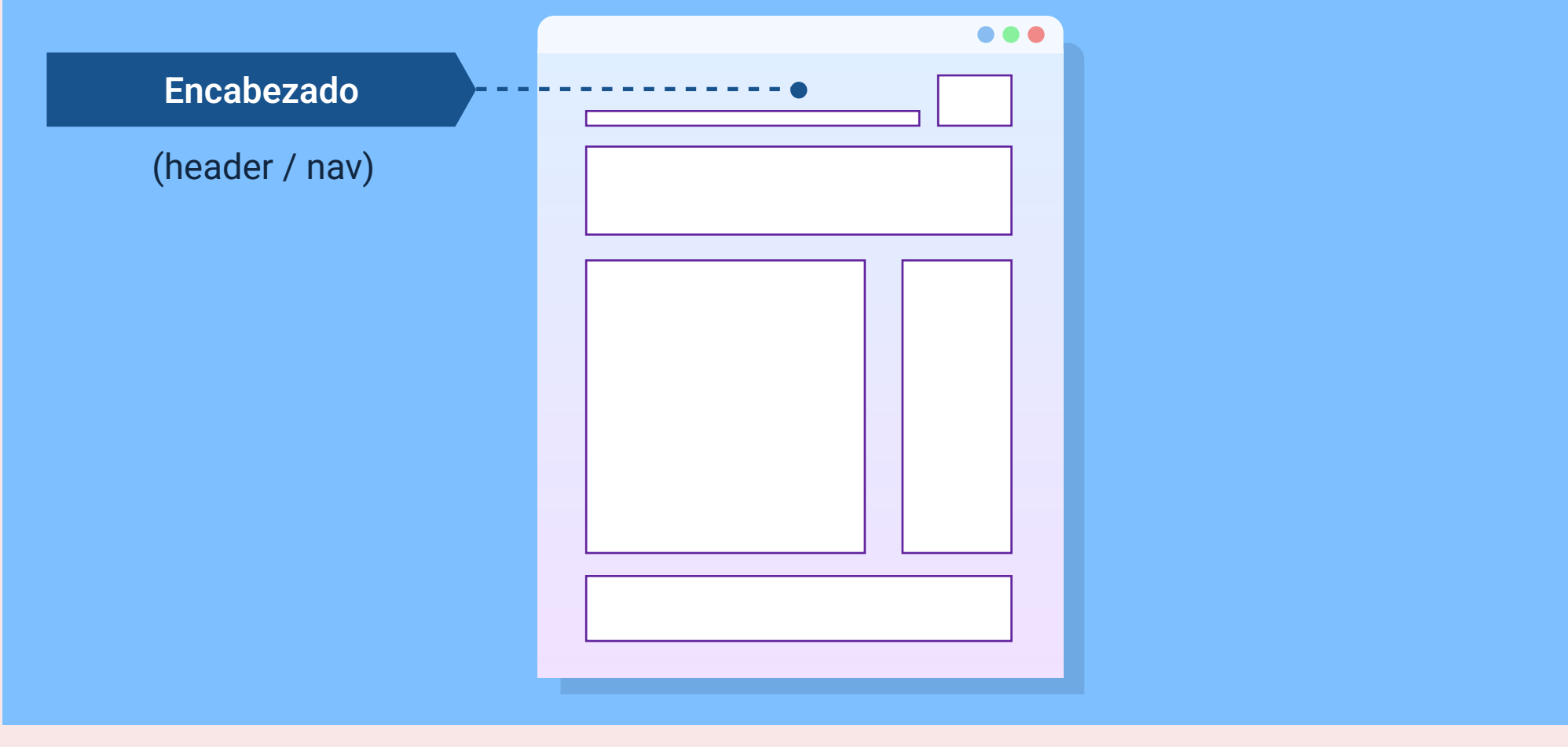
Aplicaciones web y servicios SaaS (Software as a Service)

4.3 Características de los elementos

Un sitio web se estructura mediante diferentes elementos que organizan la información y facilitan la navegación e interacción del usuario. Cada uno cumple una función específica dentro de la interfaz y contribuye a establecer una jerarquía visual y funcional del contenido. Aunque su distribución puede variar según el propósito y diseño del sitio, generalmente se identifican elementos como los siguientes:

Encabezado (header / nav)

Se ubica generalmente en la parte superior y reúne elementos de identificación y navegación, como el logotipo y el menú principal.



4.4 Requerimientos funcionales y no funcionales

La ingeniería de requerimientos permite identificar, analizar y documentar las necesidades que debe satisfacer un sitio o aplicación web. Durante este proceso se establecen las funciones que deberá ejecutar el sistema, las condiciones necesarias para su operación y los criterios que permitirán comprobar posteriormente si la solución desarrollada responde a las necesidades definidas. Los requerimientos pueden agruparse principalmente en funcionales y no funcionales:



Requerimientos funcionales (RF)

Definen los comportamientos, servicios e interacciones que el sistema debe proporcionar en respuesta a las acciones del usuario o a determinados eventos. Pueden incluir funciones como registrar usuarios, autenticar credenciales, realizar búsquedas, procesar formularios, gestionar información o ejecutar transacciones. Por ejemplo, un requerimiento funcional puede establecer: «El sistema debe permitir al usuario recuperar su contraseña mediante el correo electrónico registrado».



Requerimientos no funcionales (RNF)

Establecen las condiciones y atributos de calidad bajo los cuales deben ejecutarse las funcionalidades del sistema. Incluyen aspectos relacionados con rendimiento, seguridad, disponibilidad, accesibilidad, compatibilidad, escalabilidad y usabilidad. Por ejemplo, un requerimiento no funcional puede definir un tiempo máximo de respuesta, un porcentaje de disponibilidad o un determinado criterio de accesibilidad.

Tanto los requerimientos funcionales como los no funcionales deben formularse de manera clara, verificable y, cuando corresponda, medible, de modo que puedan establecerse criterios objetivos de aceptación. Por esta razón, expresiones ambiguas como «el sitio debe ser rápido» deben sustituirse por criterios cuantificables asociados al rendimiento, indicando la métrica, el valor esperado y las condiciones en las que será evaluada.

4.5 Parámetros de configuración, descripción y palabras clave

Los parámetros de configuración de un sitio web permiten establecer propiedades que influyen en su correcta visualización, identificación, compatibilidad y presentación de información ante navegadores y otros servicios digitales. Estas configuraciones pueden definirse tanto en el documento HTML como en otros componentes de la aplicación, dependiendo de las características técnicas y los requerimientos establecidos para el proyecto.



En HTML, una parte de esta configuración se incorpora en el elemento <head>, donde se incluyen metadatos que proporcionan información sobre el documento sin formar parte directamente del contenido visible de la página. Estos datos permiten especificar aspectos como la codificación de caracteres, la adaptación a diferentes tamaños de pantalla, la descripción del contenido, la autoría y la información utilizada al compartir el sitio en determinadas plataformas.

El siguiente fragmento de código ejemplifica la configuración de algunos de estos parámetros mediante etiquetas <meta>, propiedades Open Graph y el elemento <title>:



```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Componente formativo sobre fundamentos de desarrollo web, arquitecturas cliente-servidor y protocolos de comunicación para aprendices SENA.">
<meta name="keywords" content="desarrollo web, HTML5, CSS3, JavaScript, interoperabilidad, protocolos, SENA">
<meta name="author" content="Servicio Nacional de Aprendizaje - SENA">

<!-- Protocolo Open Graph para redes sociales -->
<meta property="og:title" content="Fundamentos para el desarrollo de aplicaciones web">
<meta property="og:description" content="Aprenda las técnicas de interoperabilidad y protocolos técnicos en desarrollo web.">
<meta property="og:type" content="website">

<title>Fundamentos para el Desarrollo de Aplicaciones Web | SENA</title>
</head>
```

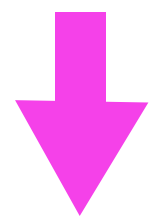
En este ejemplo, los parámetros incluidos en el <head> permiten definir información básica para la interpretación y presentación del documento web. Sin embargo, la configuración técnica de un sitio también debe responder a los requerimientos establecidos durante su desarrollo, de manera que aspectos funcionales, de seguridad, rendimiento y capacidad puedan verificarse mediante criterios de aceptación definidos. En relación con los requerimientos abordados previamente, la siguiente tabla presenta algunos ejemplos y sus respectivas métricas de aceptación:



Tabla 4. Clasificación de requerimientos de un sitio web

Categoría	Tipo de requerimiento	Descripción técnica	Métrica de aceptación
Funcional.	Autenticación.	Inicio de sesión con correo institucional.	Acceso autorizado con credenciales válidas y rechazo con credenciales inválidas.

Tablet

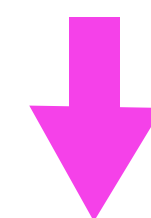


Fundamentos para el desarrollo de aplicaciones web

Este componente es una guía técnica y pedagógica para desarrollar competencias en maquetación, estructuración y despliegue de soluciones web. En diferentes unidades temáticas aborda fundamentos de arquitectura web, comunicación cliente-servidor (client-server), desarrollo frontend y backend, especificación de requerimientos técnicos y analítica de información, aplicando estándares de interoperabilidad y protocolos de red para la mejora continua del software.

Iniciar >

Movil



Fundamentos para el desarrollo de aplicaciones web

Este componente es una guía técnica y pedagógica para desarrollar competencias en maquetación, estructuración y despliegue de soluciones web. En diferentes unidades temáticas aborda fundamentos de arquitectura web, comunicación cliente-servidor (client-server), desarrollo frontend y backend, especificación de requerimientos técnicos y analítica de información, aplicando estándares de interoperabilidad y protocolos de red para la mejora continua del software.

Iniciar >



Referencias bibliográficas

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris eu tincidunt erat. Fusce cursus mauris in nulla egestas vulputate. Phasellus viverra accumsan libero, sed luctus diam maximus euismod. Mauris sem risus, tincidunt vel libero in, lobortis commodo ante. Sed pulvinar luctus orci in rhoncus. Nulla ut dapibus risus, ac tempor est. Donec nulla augue, vehicula quis malesuada vitae, fermentum non nisi. Donec tincidunt nibh a magna bibendum rutrum.

Técnico en Procesamiento de pruebas de software

Síntesis: Fundamentos para el desarrollo de aplicaciones web



La construcción de sitios web bajo criterios de interoperabilidad y protocolos técnicos requiere integrar la infraestructura tecnológica, el diseño orientado a la usabilidad y el análisis continuo de información.

El proceso inicia con una arquitectura que define la estructura y el licenciamiento del sistema. Posteriormente, se abordan los protocolos de comunicación y las técnicas de diseño responsivo, procurando que la interfaz responda a criterios de accesibilidad y rendimiento. El desarrollo requiere el uso de herramientas de edición, construcción y control de versiones. Una vez publicado el sitio web bajo los parámetros de configuración correspondientes, el análisis de información permite recopilar e interpretar métricas que orientan la identificación de oportunidades de mejora y favorecen un ciclo continuo de optimización técnica y funcional.

